



APRENDIZAGEM DE MÁQUINA

Otávio Calaça Xavier



INF

INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO
Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

Aula 8

Redes Neurais e *Deep Learning*

INF

INSTITUTO DE
INFORMÁTICA



UFMG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

Tribunal de Justiça do Estado de Goiás



Programa de
Residência em TI

AGENDA

- Introdução às Redes Neurais
- Perceptron
- Multilayer Perceptron
- Máquina Restrita de Boltzmann e Deep Belief Network
- *Deep Learning*

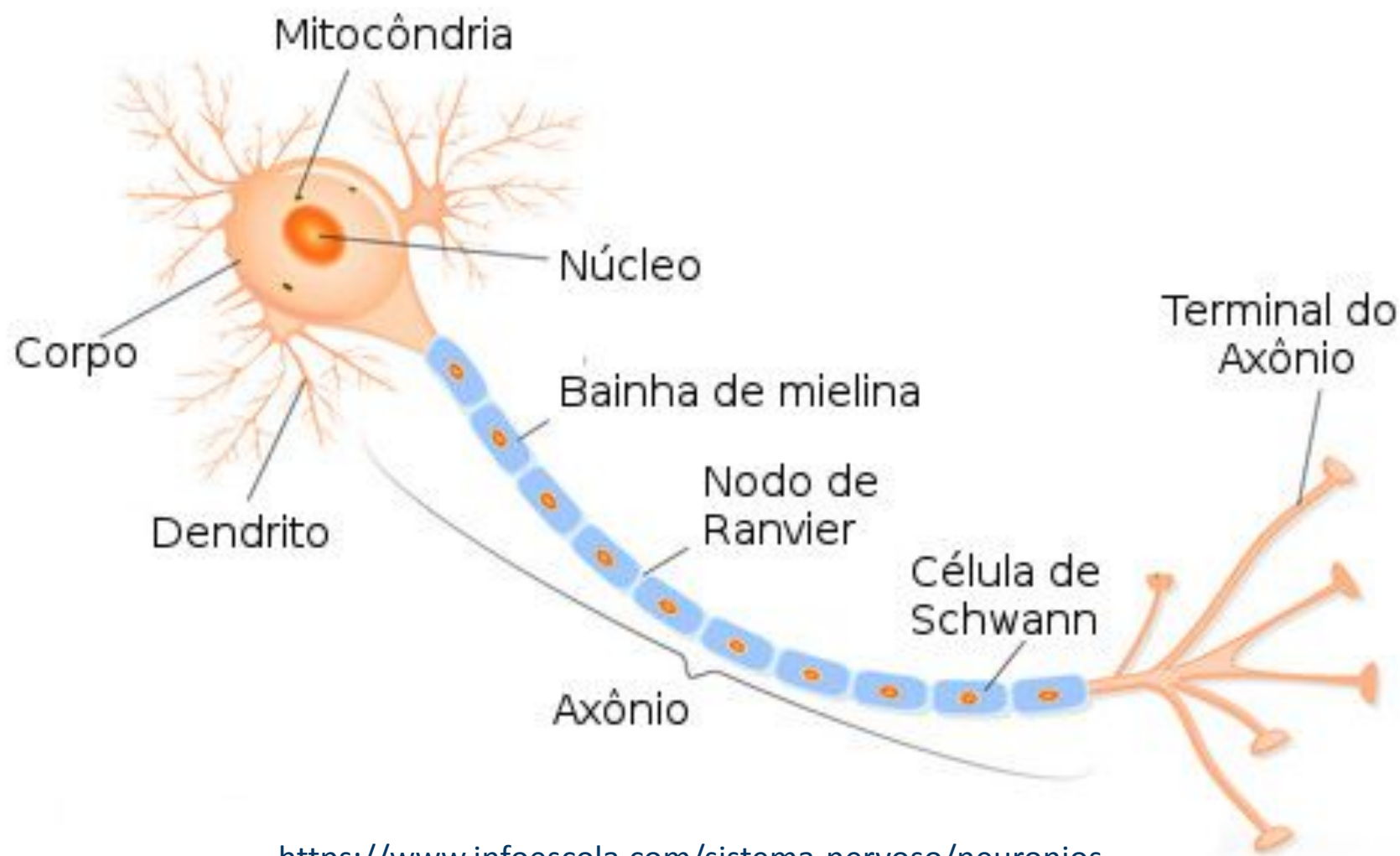
Introdução às Redes Neurais

- Uma rede neural artificial simula o comportamento de uma rede neural biológica.
- A menor unidade de uma rede neural é o neurônio.
- Cada neurônio possui diversas entradas e uma saída.

Introdução às Redes Neurais

- A saída é ativada de acordo com as entradas.
- As entradas são ponderadas.
- A ponderação entre as entradas é adaptativa.
- Essa adaptação garante a plasticidade do sistema nervoso.
- Aprendizagem consiste em modificar a ponderação das entradas.

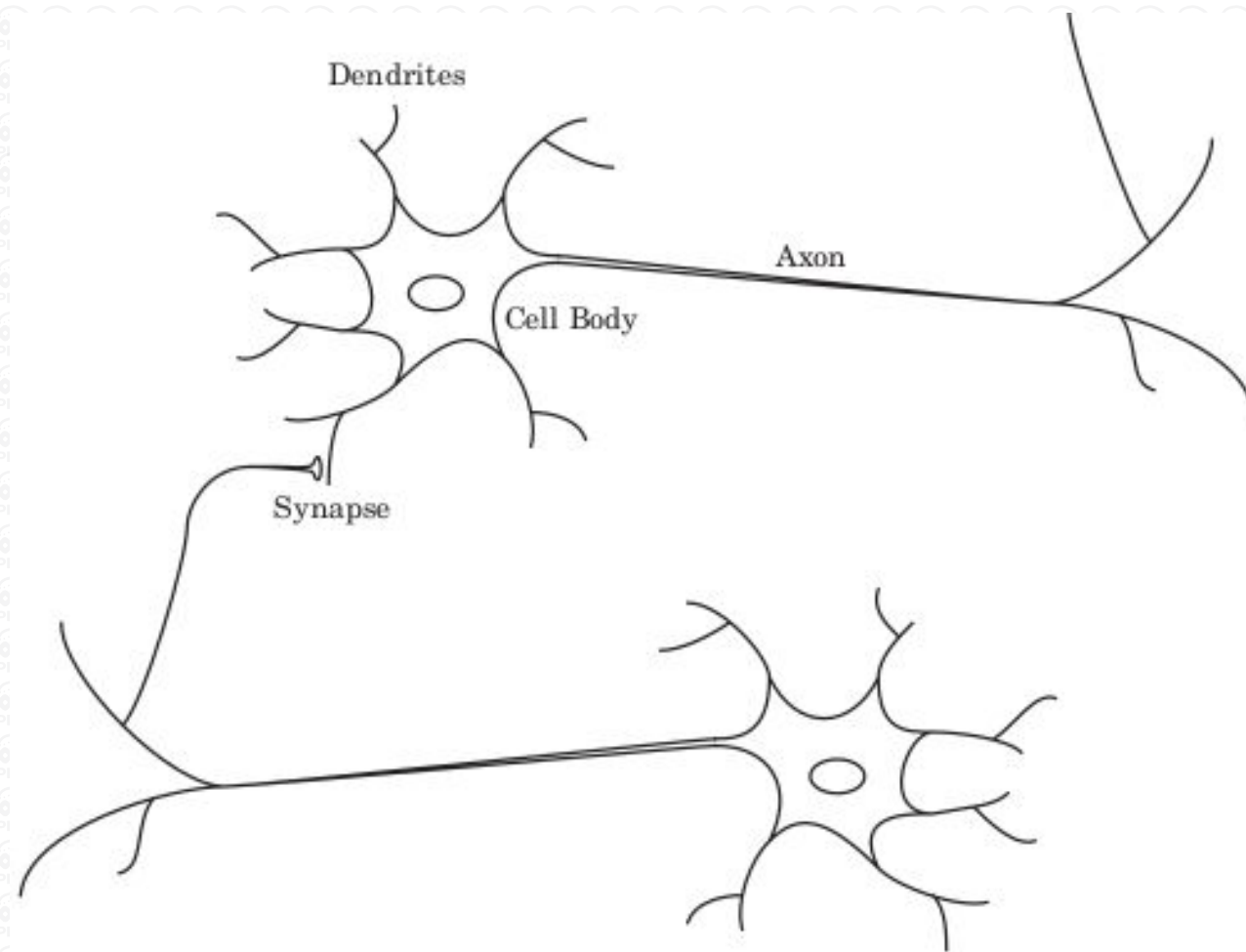
O Neurônio Biológico



O Neurônio Biológico

- Dendritos são as entradas.
- Axônio é a saída.
- Sinapse é o local onde ocorre a comunicação entre neurônios.
- A comunicação é motivada por neurotransmissores.

O Neurônio Biológico



O Neurônio Biológico - Núcleo

- O núcleo do neurônio é responsável por controlar o sinal no axônio.
- Realiza uma ponderação entre as entradas para retornar um sinal de saída.
- Matematicamente pode ser representado por uma soma ponderada e uma função de ativação.

O Neurônio Biológico - Processamento

- O neurônio biológico tem alta capacidade de processamento paralelo.
- O sistema nervoso de um ser humano tem cerca de 100 bilhões de neurônios.
- Cada neurônio pode fazer até 10 mil conexões.

Redes Neurais Biológicas

- **Verme** C. Elegans tem **400** neurônios
- **Formiga** tem **250 mil** neurônios
- **Barata** tem **1 milhão** de neurônios
- **Ratos** possuem **200 milhões** de neurônios
- **Humanos** possuem cerca de **100 bilhões** de neurônios.



Redes Neurais Artificiais - Histórico

- Antes de 1940: von Hemholtz, Mach, Pavlov, etc.
 - Teorias gerais sobre treinamento, visão e condicionamento.
 - Nenhum modelo matemático para um neurônio.
- Década de 1940: Hebb, McCulloch e Pitts
 - Mecanismo de aprendizado em neurônios biológicos.
 - Primeiras representações de neurônio artificial: perceptron.
- Década de 1950: Rosenblatt, Widrow e Hoff
 - Primeiras redes neurais práticas e regras de aprendizagem.
 - Adaline e Regra Delta.

Redes Neurais Artificiais - Histórico

- Década de 1960: Minsky e Papert
 - Demonstração de limitações nas redes neurais existentes, novos algoritmos de aprendizagem são desencorajados e algumas pesquisas são suspensas.
- Década de 1970: Amari, Anderson, Fukushima, Grossberg, Kohonen
 - Progresso com ritmo lento.
- Década de 1980: Grossberg, Hopfield, Kohonen, Rumelhart, etc.
 - Novos desenvolvimentos importantes causam um ressurgimento desse campo de pesquisa.

Redes Neurais Artificiais - Histórico (Anos 90 - 2000)



- **A Era das SVMs:** Máquinas de Vetores de Suporte (SVM) e *Random Forests* ganham popularidade por serem matematicamente mais robustas e exigirem menos poder computacional que as redes neurais da época.
- **O problema do Desvanecimento do Gradiente:** Dificuldade em treinar redes com muitas camadas (profundas).
- **Yann LeCun (1998):** Aplicação prática de CNNs (LeNet) para reconhecimento de dígitos manuscritos (MNIST), mantendo a chama das redes neurais acesa.

Redes Neurais Artificiais - Histórico (2010 aos Dias Atuais (O Boom))



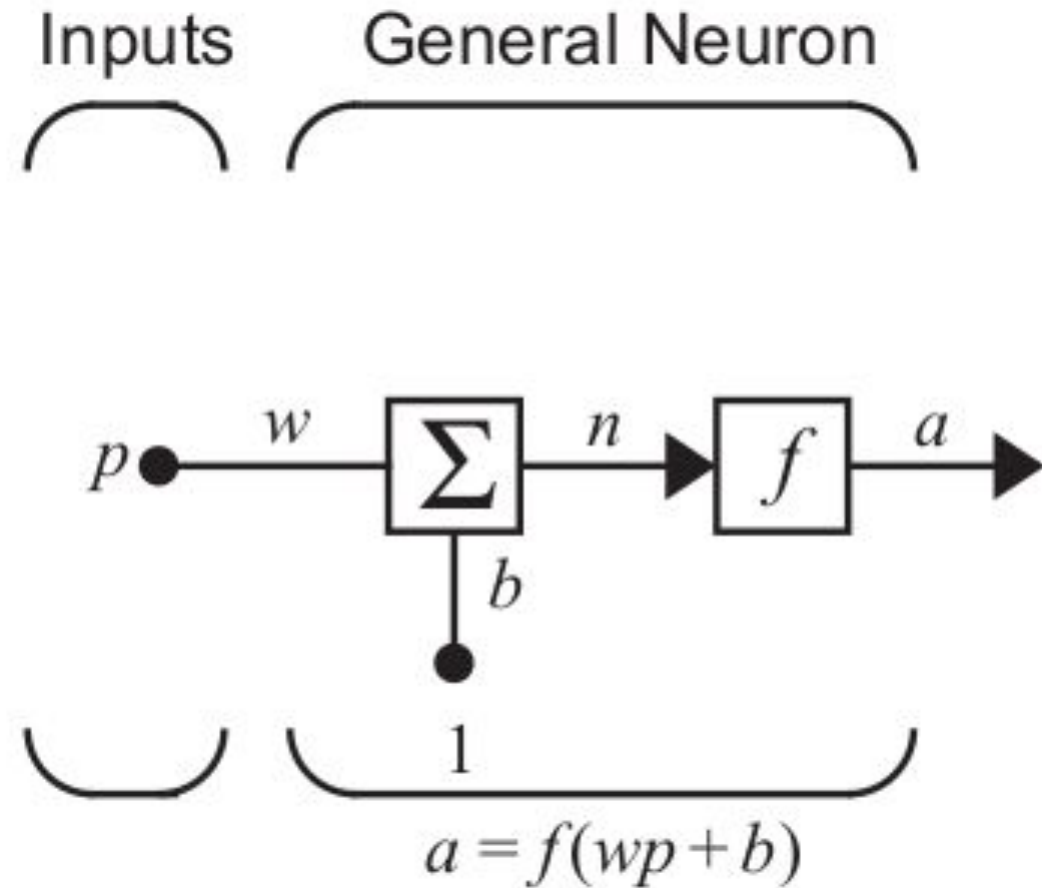
- **2012 (AlexNet):** Momento de ruptura. Uma rede convolucional vence a competição ImageNet com larga margem, provando a superioridade do Deep Learning para visão computacional.
- **Hardware (GPUs):** Uso de placas de vídeo (NVIDIA) para processamento paralelo massivo, permitindo o treinamento de redes gigantes.
- **Big Data:** Disponibilidade massiva de dados para treinamento.
- **2017 (Transformers):** O artigo "Attention Is All You Need" (Google) introduz a arquitetura Transformer, base para o ChatGPT, BERT e LLaMA.

Redes Neurais Artificiais - Aplicações

- Aeroespacial e Automóveis
- Bancos
- Defesa
- Eletrônica
- Financeira
- Fábricas/Indústrias
- Medicina
- entre outros ...

Representação Matemática de um Neurônio - Perceptron

p => entrada ou vetor de entradas
 w => peso ou vetor de pesos
 b => bias ou limiar
 n => resultado do somatório
 f => função de ativação
 a => sinal de saída



<http://hagan.ecen.ceat.okstate.edu/nnd.html>

Perceptron

- A rede perceptron possui apenas uma camada
- É uma abordagem supervisionada
- O aprendizado se dá pelo ajuste dos pesos
- A função de ativação é do tipo limiar

Perceptron - Funções de Ativação

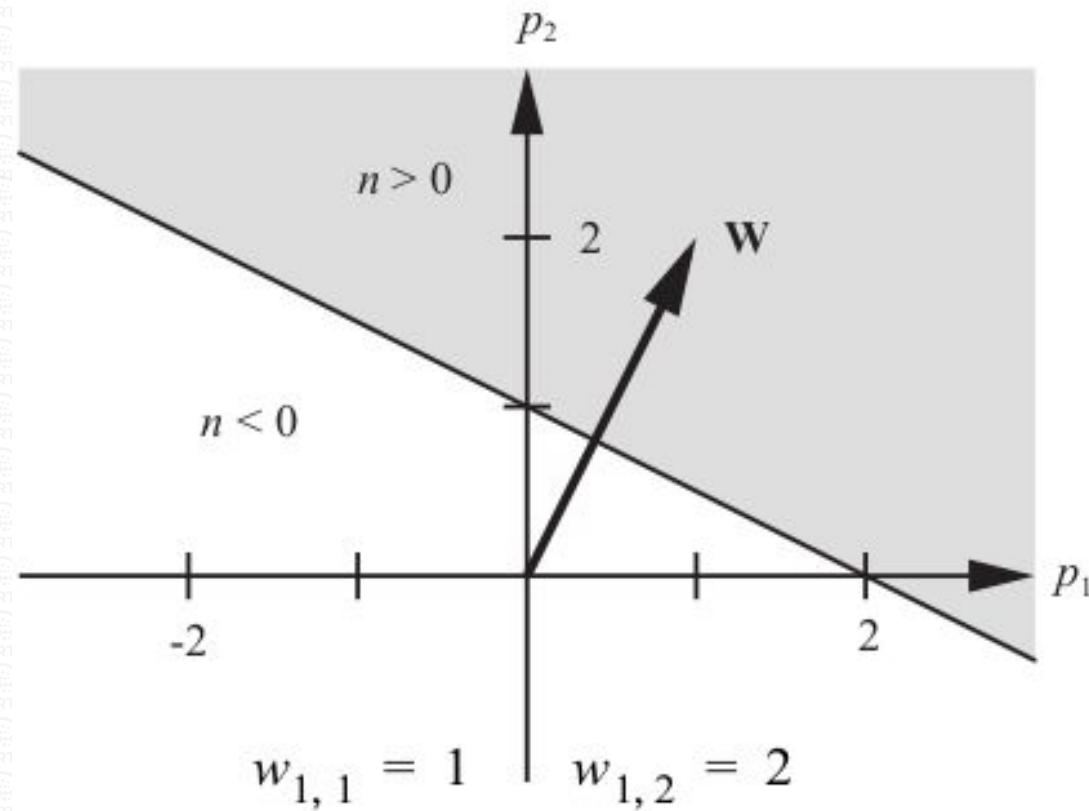
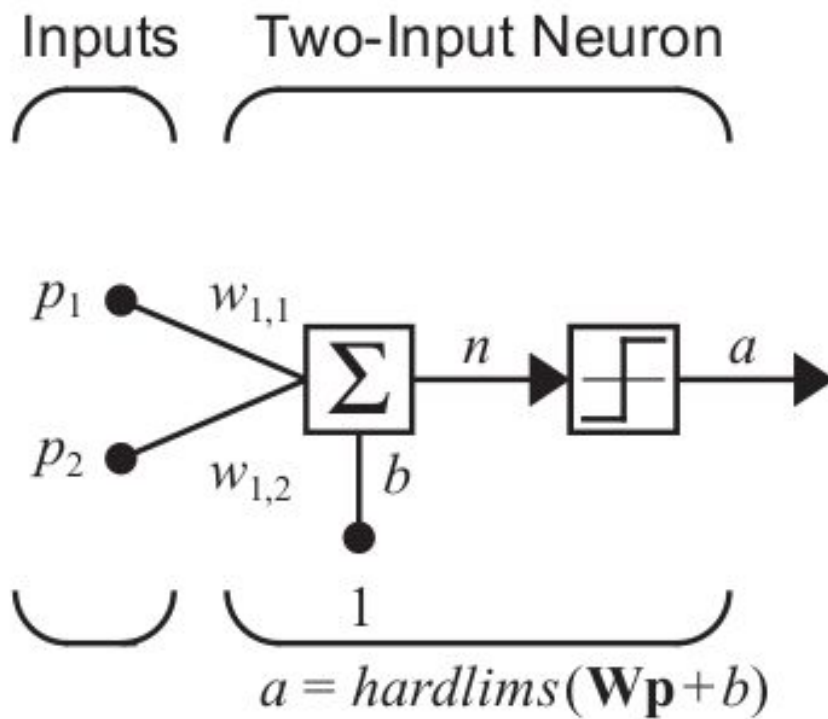
Name	Input/Output Relation	Icon
Symmetric Saturating Linear	$a = -1 \quad n < -1$ $a = n \quad -1 \leq n \leq 1$ $a = 1 \quad n > 1$	
Log-Sigmoid	$a = \frac{1}{1 + e^{-n}}$	
Hyperbolic Tangent Sigmoid	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$	
Positive Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n$	
Competitive	$a = 1 \quad \text{neuron with max } n$ $a = 0 \quad \text{all other neurons}$	

Name	Input/Output Relation	Icon
Hard Limit	$a = 0 \quad n < 0$ $a = 1 \quad n \geq 0$	
Symmetrical Hard Limit	$a = -1 \quad n < 0$ $a = +1 \quad n \geq 0$	
Linear	$a = n$	
Saturating Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n \leq 1$ $a = 1 \quad n > 1$	

Perceptron - Funções de Ativação (ReLU)

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$.
- **Vantagem:** Resolve o problema do desvanecimento do gradiente (vanishing gradient) que afetava a Sigmoid e a Tangente Hiperbólica em redes profundas.
- **Eficiência:** Computacionalmente muito leve (apenas verifica se o valor é positivo).
- **Variações:** Leaky ReLU, ELU (para evitar "neurônios mortos").

Perceptron



$$n = w_{1,1}p_1 + w_{1,2}p_2 + \dots + w_{1,R}p_R + b.$$

Perceptron

Os pesos são ajustados pela seguinte equação:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - f(x_i))$$

$w_j(t)$ é o peso atual da entrada j

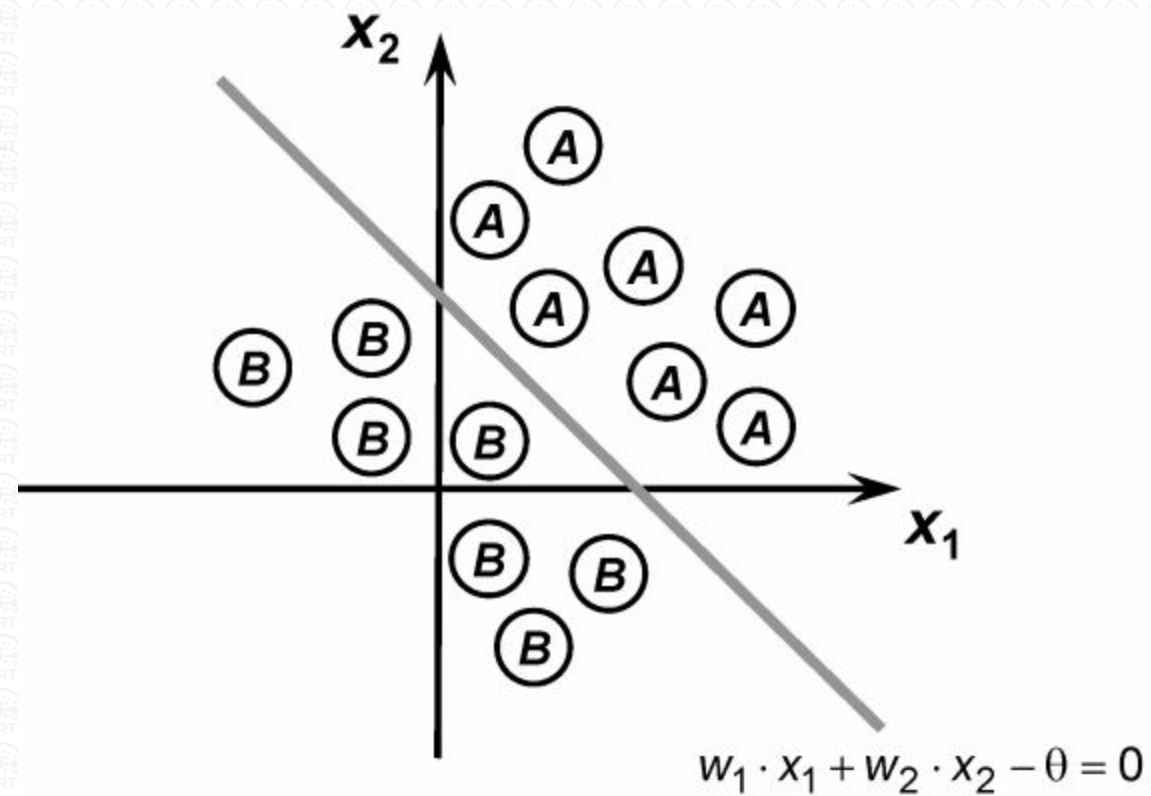
η é a taxa de aprendizado

x_i^j é o valor da entrada que estimula o peso avaliado

y_i é o valor desejado

$f(x_i)$ é o valor obtido na iteração atual

Perceptron



Perceptron

Início {Algoritmo *Perceptron* – Fase de Treinamento}

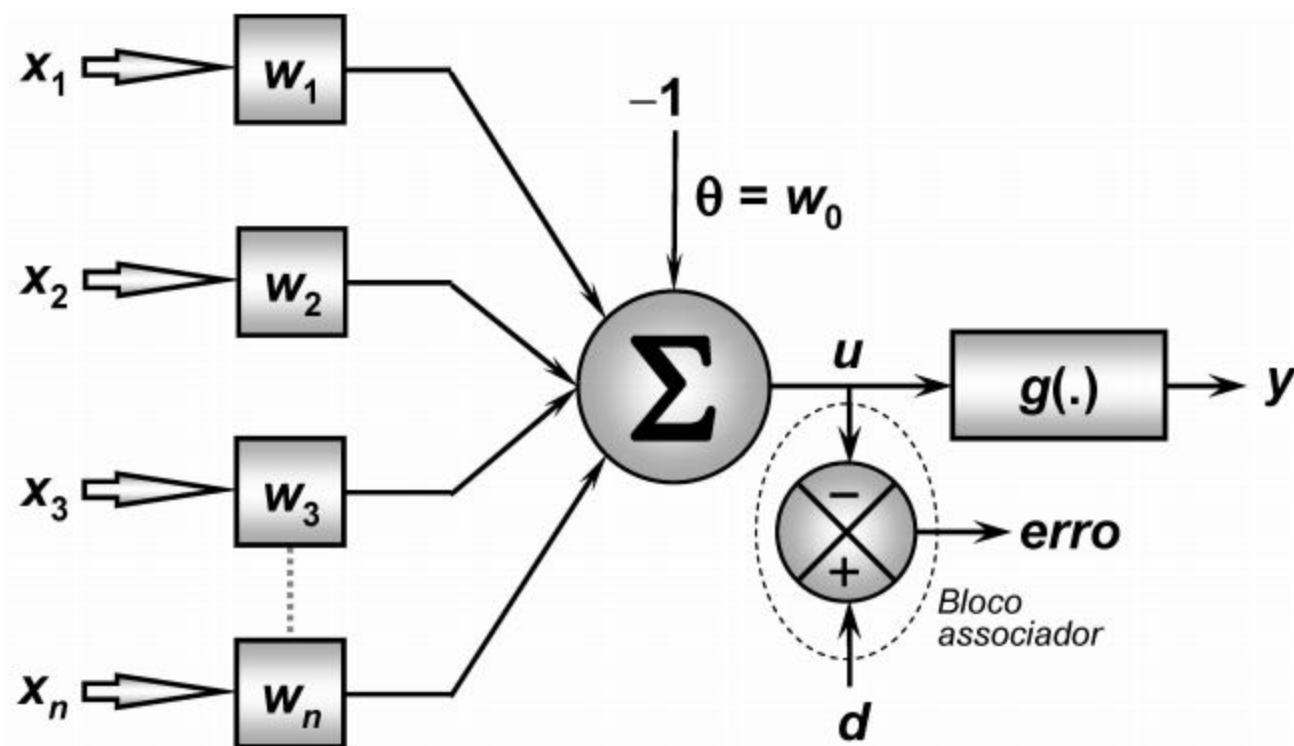
- <1> Obter o conjunto de amostras de treinamento $\{x^{(k)}\}$;
 - <2> Associar a saída desejada $\{d^{(k)}\}$ para cada amostra obtida;
 - <3> Iniciar o vetor w com valores aleatórios pequenos;
 - <4> Especificar a taxa de aprendizagem $\{\eta\}$;
 - <5> Iniciar o contador de número de épocas $\{\text{época} \leftarrow 0\}$;
 - <6> Repetir as instruções:
 - <6.1> erro $\leftarrow$ “inexiste”;
 - <6.2> Para todas as amostras de treinamento $\{x^{(k)}, d^{(k)}\}$, fazer:
 - <6.2.1> $u \leftarrow w^T \cdot x^{(k)}$;
 - <6.2.2> $y \leftarrow \text{sinal}(u)$;
 - <6.2.3> Se $y \neq d^{(k)}$
 - <6.2.3.1> Então $\begin{cases} w \leftarrow w + \eta \cdot (d^{(k)} - y) \cdot x^{(k)} \\ \text{erro} \leftarrow \text{“existe”} \end{cases}$
 - <6.3> $\text{época} \leftarrow \text{época} + 1$;
- Até que: erro $\leftarrow$ “inexiste”

Fim {Algoritmo *Perceptron* – Fase de Treinamento}

ADALINE

- *ADaptative Llinear NEuron* – Neurônio Linear Adaptativo
- Desenvolvida por Ted Hoff e Bernard Widrow na década de 1960.
- O erro é calculado antes de passar pela função de ativação.
- Desenvolvimento do algoritmo de aprendizado denominado Regra Delta.

ADALINE



ADALINE - Treinamento

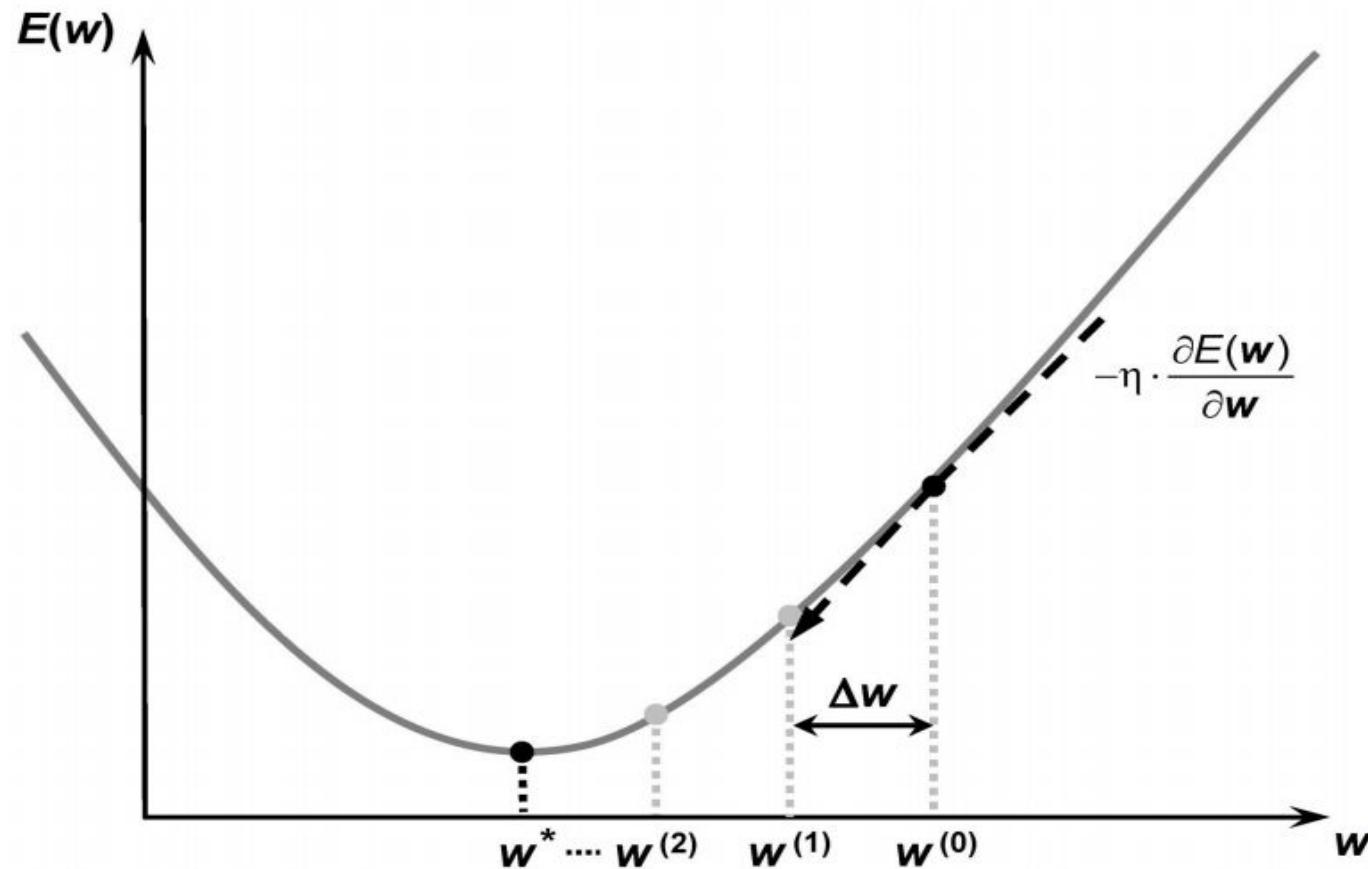
Início {Algoritmo *Adaline* – Fase de Treinamento}

- <1> Obter o conjunto de amostras de treinamento $\{ \mathbf{x}^{(k)} \}$;
 - <2> Associar a saída desejada $\{ d^{(k)} \}$ para cada amostra obtida;
 - <3> Iniciar o vetor $\mathbf{w}$ com valores aleatórios pequenos;
 - <4> Especificar taxa de aprendizagem $\{ \eta \}$ e precisão requerida $\{ \varepsilon \}$;
 - <5> Iniciar o contador de número de épocas $\{ \text{época} \leftarrow 0 \}$;
 - <6> Repetir as instruções:
 - <6.1> $E_{qm}^{anterior} \leftarrow E_{qm}(\mathbf{w})$;
 - <6.2> Para todas as amostras de treinamento $\{ \mathbf{x}^{(k)}, d^{(k)} \}$, fazer:
 - <6.2.1> $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}^{(k)}$;
 - <6.2.2> $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$;
 - <6.3> $\text{época} \leftarrow \text{época} + 1$;
 - <6.4> $E_{qm}^{atual} \leftarrow E_{qm}(\mathbf{w})$;
- Até que: $|E_{qm}^{atual} - E_{qm}^{anterior}| \leq \varepsilon$

Fim {Algoritmo *Adaline* – Fase de Treinamento}

ADALINE

A regra delta analisa o delta (diferença) entre as épocas.



Erro Quadrático Médio

ADALINE utiliza o Erro Quadrático Médio para ajuste dos pesos.

$$E_{qm}(w) = \frac{1}{p} \sum_{k=1}^p (d^{(k)} - u)^2$$

O algoritmo converge de acordo com uma precisão imposta:

$$|E_{qm}(w^{\text{atual}}) - E_{qm}(w^{\text{anterior}})| \leq \epsilon$$

ADALINE - Erro Quadrático Médio

Início {Algoritmo EQM}

- <1> Obter a quantidade de padrões de treinamento $\{p\}$;
- <2> Iniciar a variável E_{qm} com valor zero $\{E_{qm} \leftarrow 0\}$;
- <3> Para todas as amostras de treinamento $\{x^{(k)}, d^{(k)}\}$, fazer:
 - <3.1> $u \leftarrow w^T \cdot x^{(k)}$;
 - <3.2> $E_{qm} \leftarrow E_{qm} + (d^{(k)} - u)^2$;
- <4> $E_{qm} \leftarrow \frac{E_{qm}}{p}$;

Fim {Algoritmo EQM}

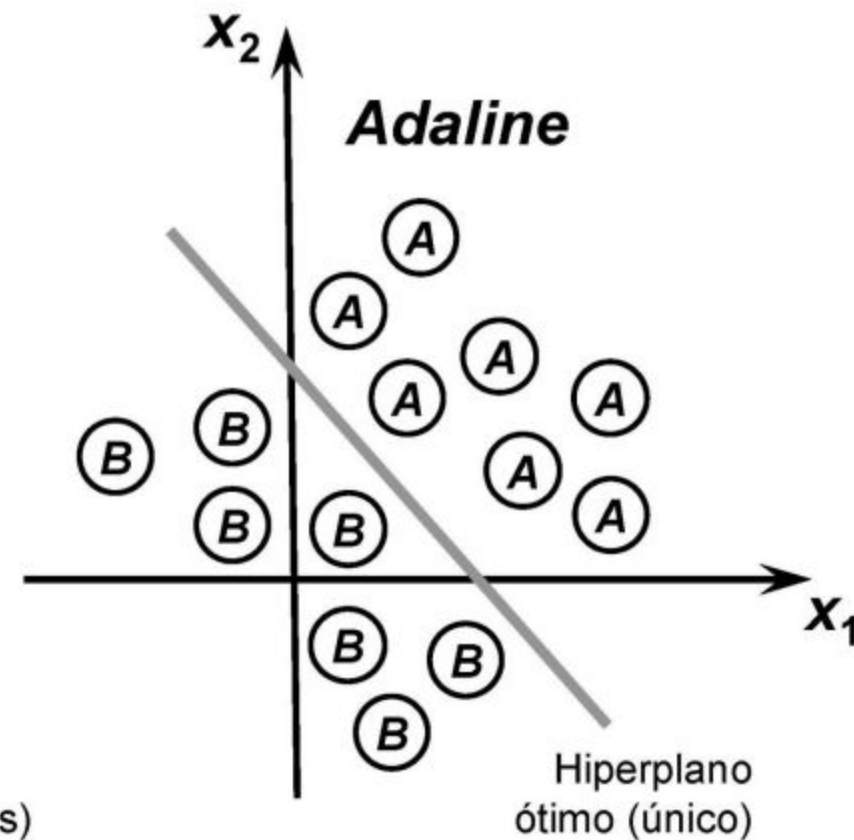
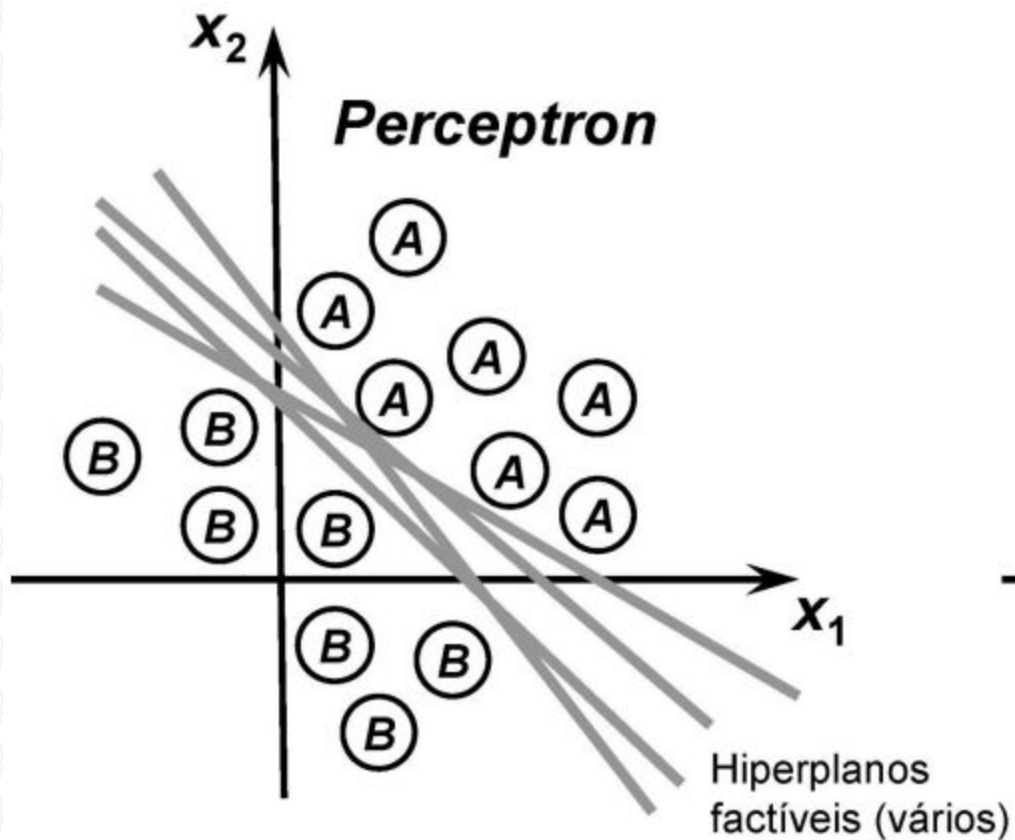
ADALINE - Operação

Início {Algoritmo *Adaline* – Fase de Operação}

- <1> Obter uma amostra a ser classificada $\{ \mathbf{x} \}$;
- <2> Utilizar o vetor $\mathbf{w}$ ajustado durante o treinamento;
- <3> Executar as seguintes instruções:
 - <3.1> $u \leftarrow \mathbf{w}^T \cdot \mathbf{x}$;
 - <3.2> $y \leftarrow \text{sinal}(u)$;
 - <3.3> Se $y = -1$
 - <3.3.1> Então: amostra $\mathbf{x} \in \{\text{Classe A}\}$
 - <3.4> Se $y = 1$
 - <3.4.1> Então: amostra $\mathbf{x} \in \{\text{Classe B}\}$

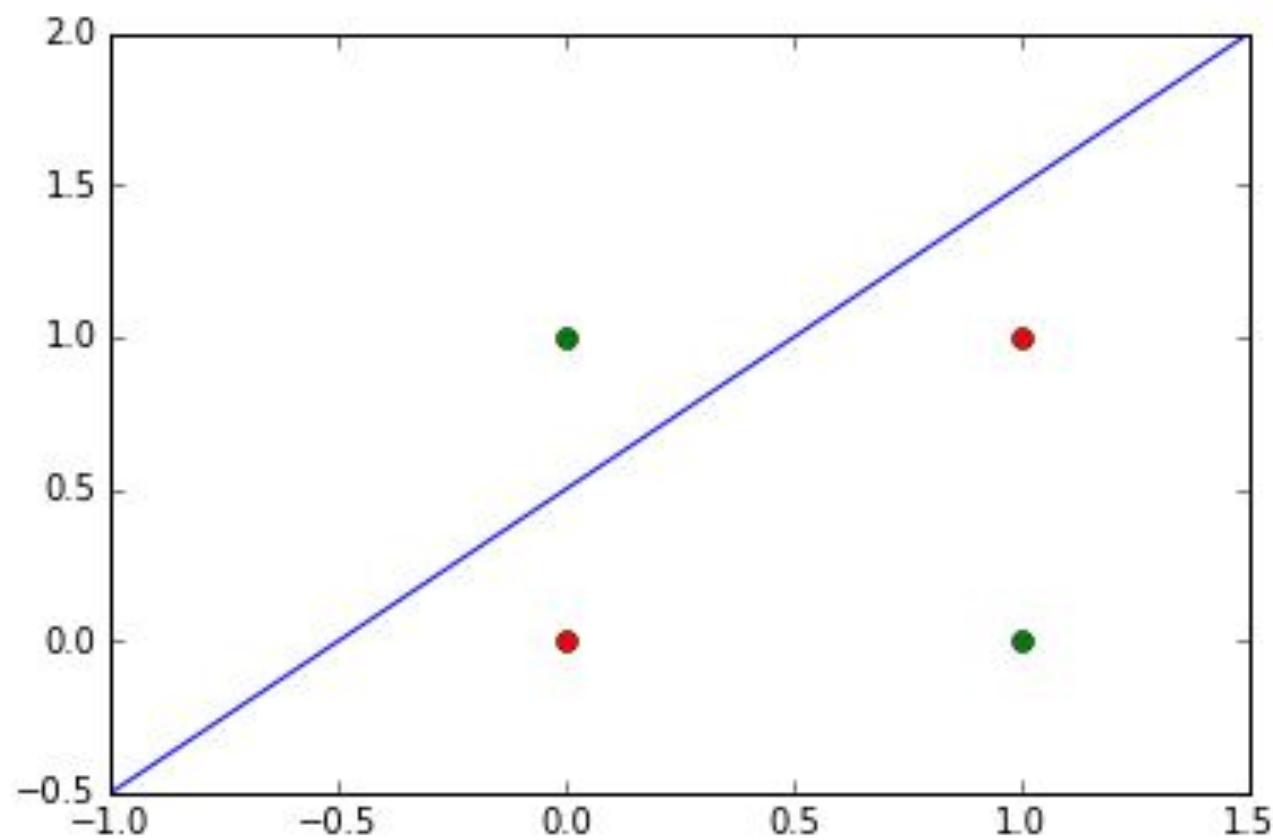
Fim {Algoritmo *Adaline* – Fase de Operação}

Comparação Perceptron e ADALINE



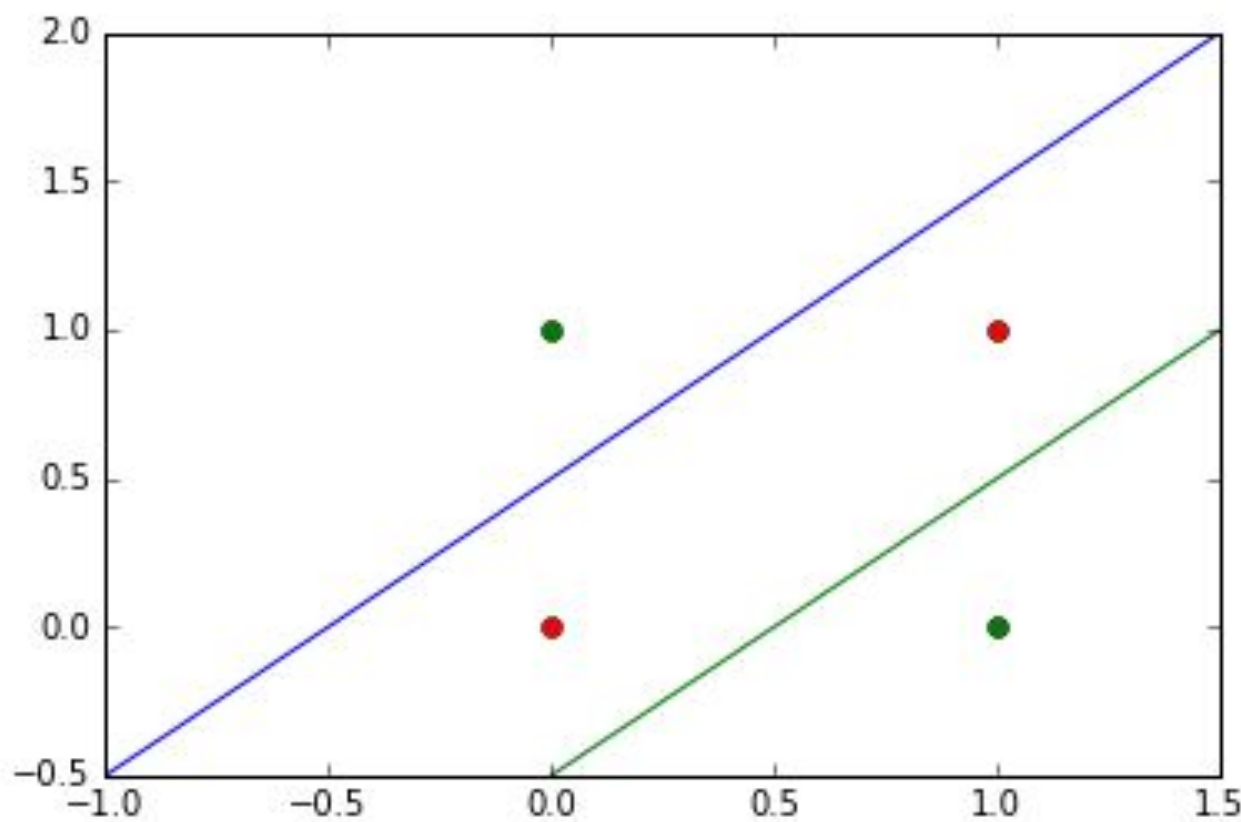
Problemas não lineares

XOR

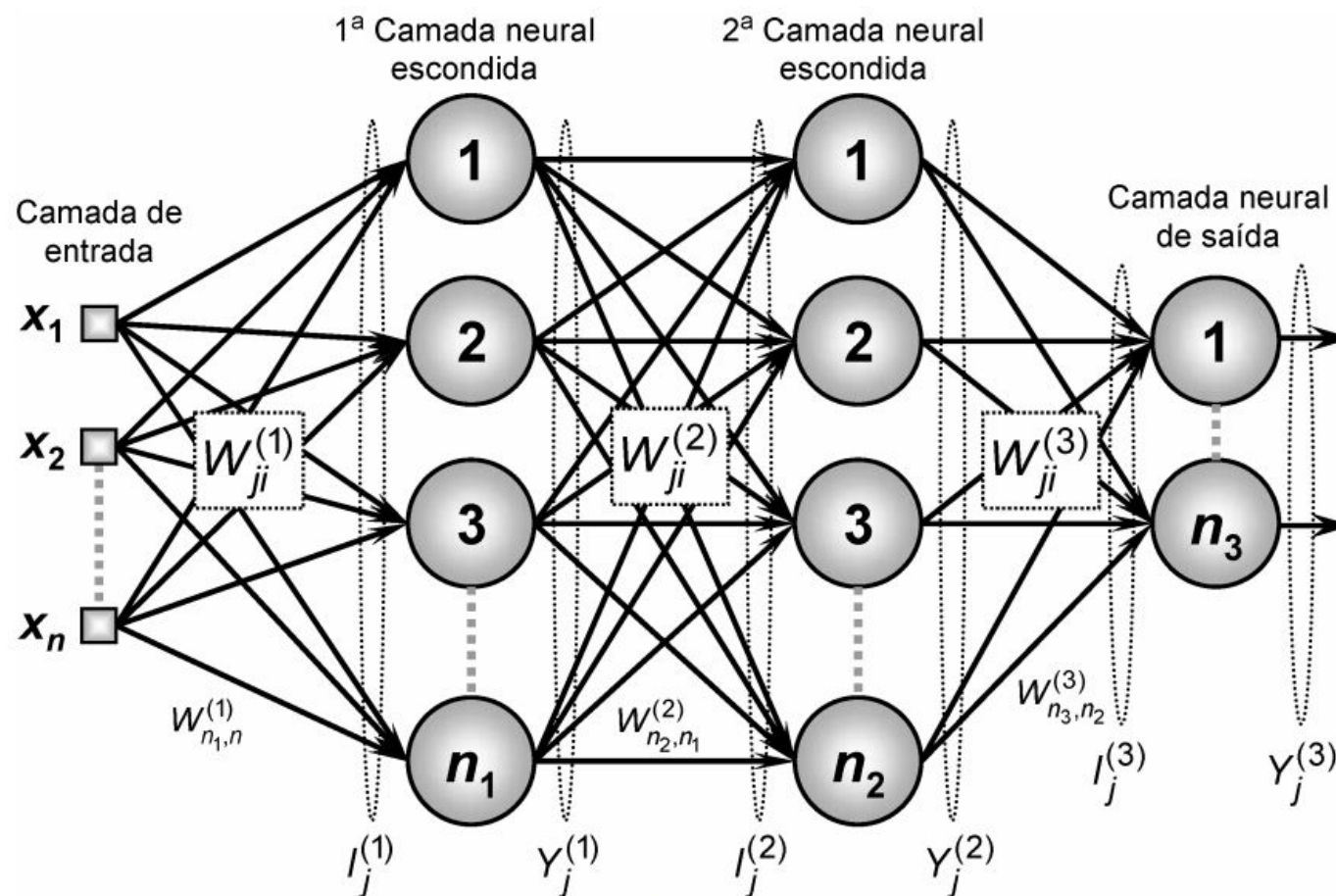


Problemas não lineares

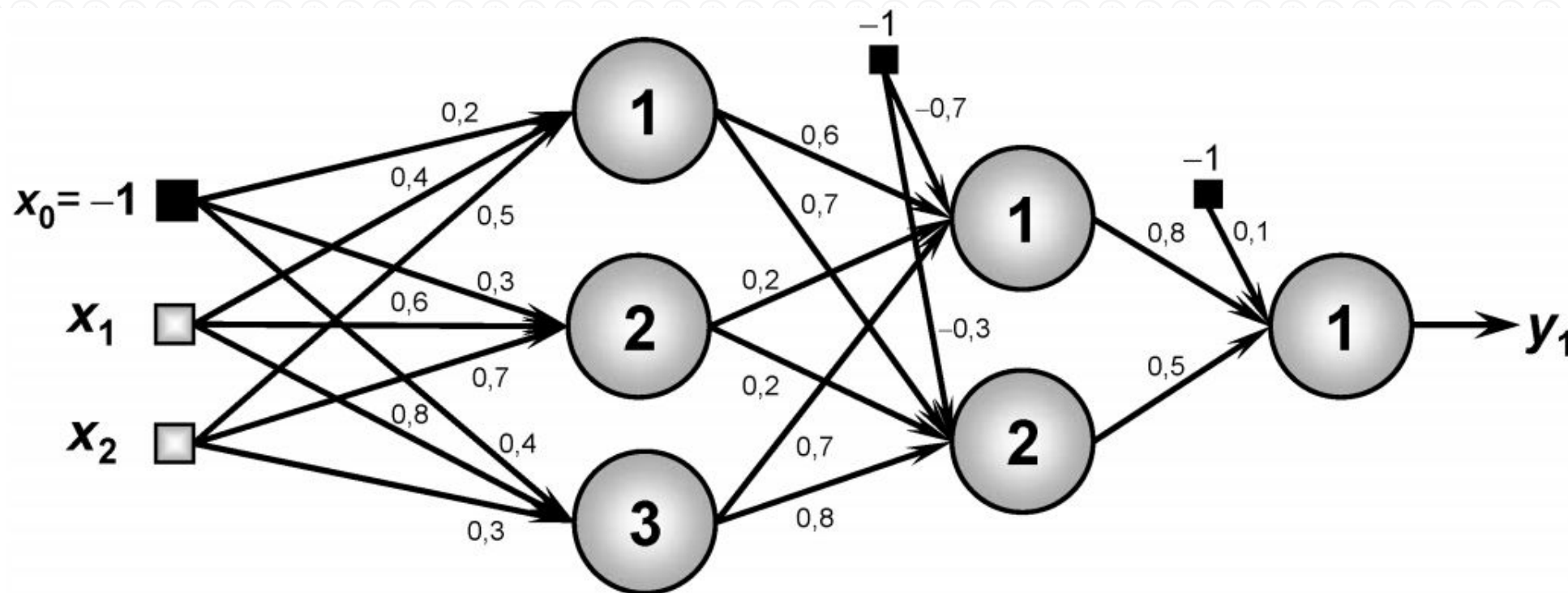
XOR



Multilayer Perceptron - MLP



Multilayer Perceptron - MLP



Multilayer Perceptron - MLP

Início {Algoritmo PMC – Fase de Operação}

- <1> Obter uma amostra $\{x\}$;
- <2> Assumir $W_{ji}^{(1)}$, $W_{ji}^{(2)}$ e $W_{ji}^{(3)}$ já ajustadas no treinamento;
- <3> Execute as seguintes instruções:
 - <3.1> Obter $I_j^{(1)}$ e $Y_j^{(1)}$; {conforme (5.1) e (5.4)}
 - <3.2> Obter $I_j^{(2)}$ e $Y_j^{(2)}$; {conforme (5.2) e (5.5)}
 - <3.3> Obter $I_j^{(3)}$ e $Y_j^{(3)}$; {conforme (5.3) e (5.6)}
- <4> Disponibilizar as saídas da rede, as quais são dadas pelos elementos contidos em $Y_j^{(3)}$

Passo
Forward

Fim {Algoritmo PMC – Fase de Operação}

Multilayer Perceptron MLP

Utiliza o
Backpropagation

Início {Algoritmo PMC – Fase de Treinamento}

- <1> Obter o conjunto de amostras de treinamento $\{x^{(k)}\}$;
 - <2> Associar o vetor de saída desejada $\{d^{(k)}\}$ para cada amostra;
 - <3> Iniciar $w_{ji}^{(1)}$, $w_{ji}^{(2)}$ e $w_{ji}^{(3)}$ com valores aleatórios pequenos;
 - <4> Especificar taxa de aprendizagem $\{\eta\}$ e precisão requerida $\{\varepsilon\}$;
 - <5> Iniciar o contador de número de épocas $\{\text{época} \leftarrow 0\}$;
 - <6> Repetir as instruções:
 - <6.1> $E_M^{\text{anterior}} \leftarrow E_M$; {conforme (5.8)}
 - <6.2> Para todas as amostras de treinamento $\{x^{(k)}, d^{(k)}\}$, fazer:
 - <6.2.1> Obter $f_j^{(1)}$ e $y_j^{(1)}$; {conforme (5.1) e (5.4)}
 - <6.2.2> Obter $f_j^{(2)}$ e $y_j^{(2)}$; {conforme (5.2) e (5.5)}
 - <6.2.3> Obter $f_j^{(3)}$ e $y_j^{(3)}$; {conforme (5.3) e (5.6)}
 - <6.2.4> Determinar $\delta_j^{(3)}$; {conforme (5.15)}
 - <6.2.5> Ajustar $w_{ji}^{(3)}$; {conforme (5.17)}
 - <6.2.6> Determinar $\delta_j^{(2)}$; {conforme (5.26)}
 - <6.2.7> Ajustar $w_{ji}^{(2)}$; {conforme (5.28)}
 - <6.2.8> Determinar $\delta_j^{(1)}$; {conforme (5.37)}
 - <6.2.9> Ajustar $w_{ji}^{(1)}$; {conforme (5.39)}
 - <6.3> Obter $y_j^{(3)}$ ajustado; {conforme <6.2.1>, <6.2.2> e <6.2.3>}
 - <6.4> $E_M^{\text{atual}} \leftarrow E_M$; {conforme (5.8)}
 - <6.5> $\text{época} \leftarrow \text{época} + 1$;
- Até que: $|E_M^{\text{atual}} - E_M^{\text{anterior}}| \leq \varepsilon$

Fim {Algoritmo PMC – Fase de Treinamento}

Deep Learning



Aprendizado Profundo

INF

INSTITUTO DE
INFORMÁTICA



UFG
UNIVERSIDADE
FEDERAL DE GOIÁS



PODER JUDICIÁRIO

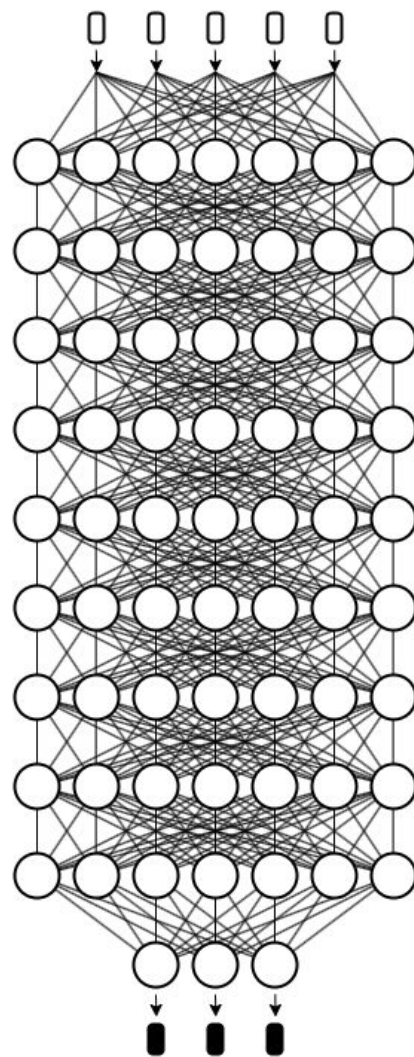
Tribunal de Justiça do Estado de Goiás



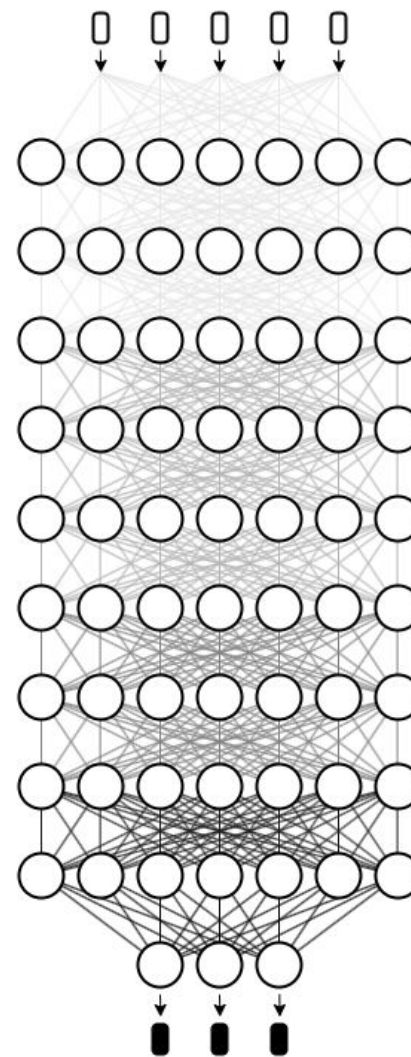
Programa de
Residência em TI

Vanishing Gradient

Good Backpropagation



Vanishing Gradients



Back-Propagation

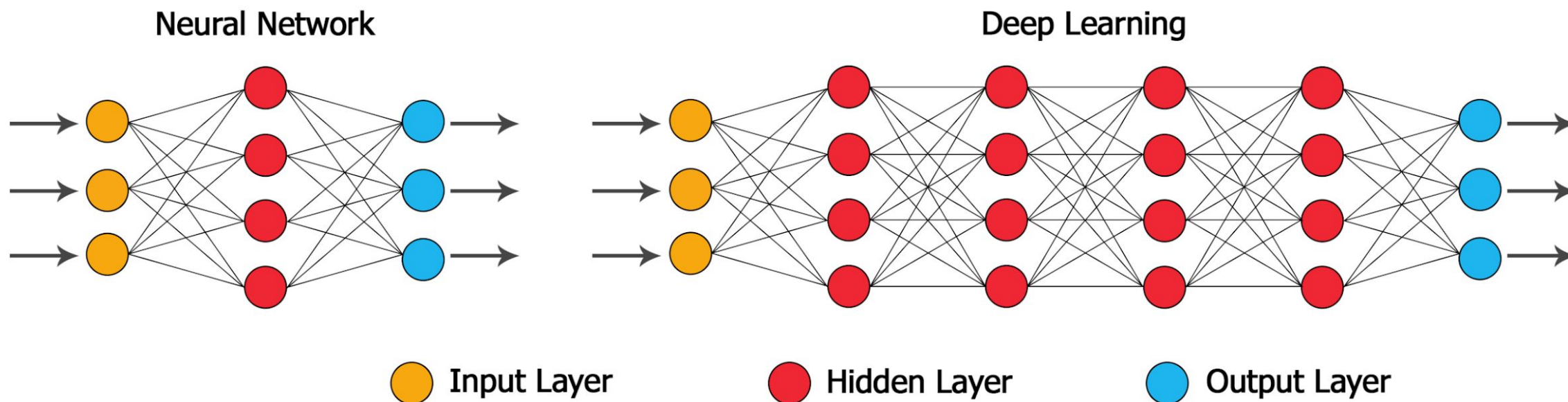


Por que *Deep Learning* funciona agora?

- **Algoritmos Aprimorados:** Funções de ativação (ReLU), otimizadores (Adam), regularização (Dropout) e novas arquiteturas (ResNet, Transformers).
- **Dados (Big Data):** Redes profundas precisam de milhões de exemplos para aprender, algo que a internet proporcionou (ex: ImageNet, Common Crawl).
- **Poder Computacional:** O treinamento que levava meses em CPUs nos anos 90 agora é feito em horas/dias com GPUs e TPUs modernas.

Deep Learning

- redes neurais artificiais de muitas camadas.
- modela abstrações de alto nível em dados.



Redes Neurais Convolucionais

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

=

6		

$$7 \times 1 + 4 \times 1 + 3 \times 1 + 2 \times 0 + 5 \times 0 + 3 \times 0 + 3 \times -1 + 3 \times -1 + 2 \times -1 = 6$$



Original



$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

=



Blur (with a mean filter)

Input image



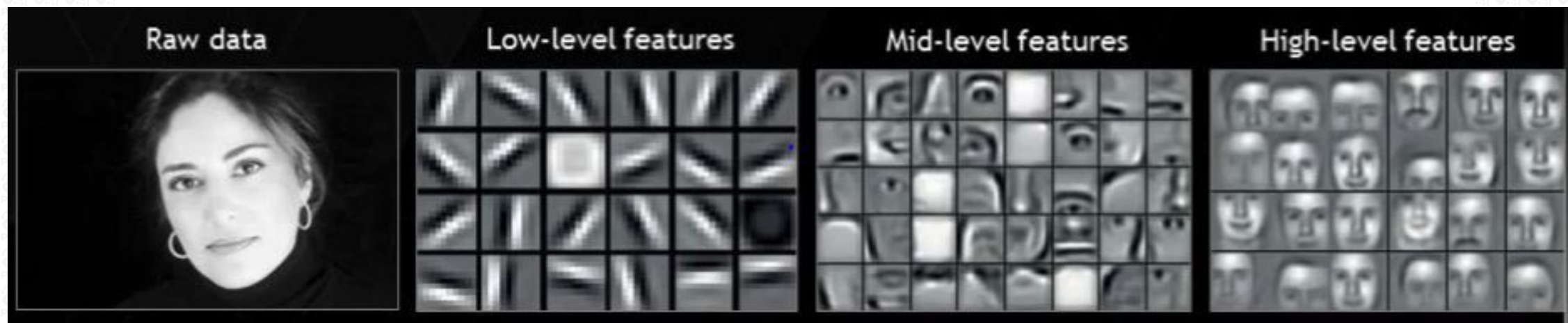
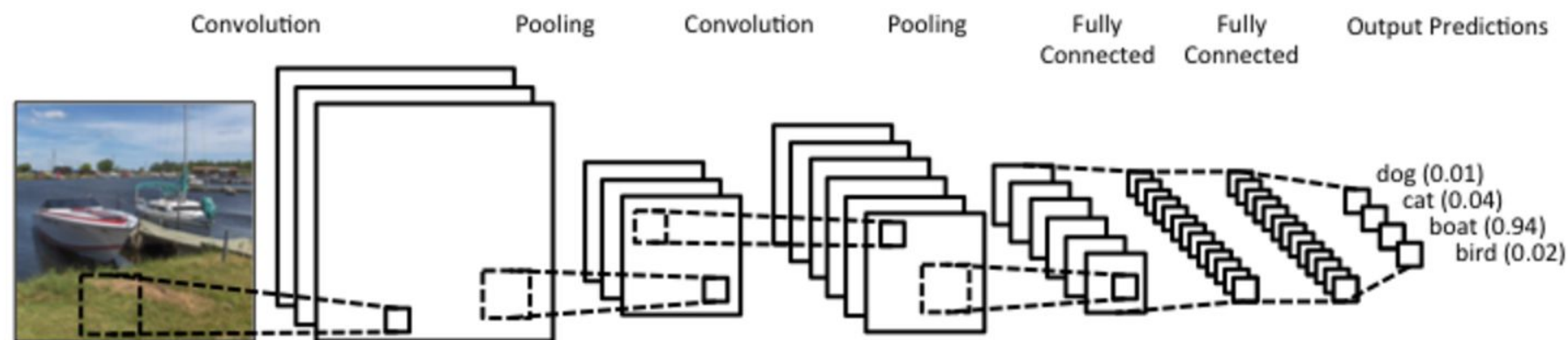
Convolution Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

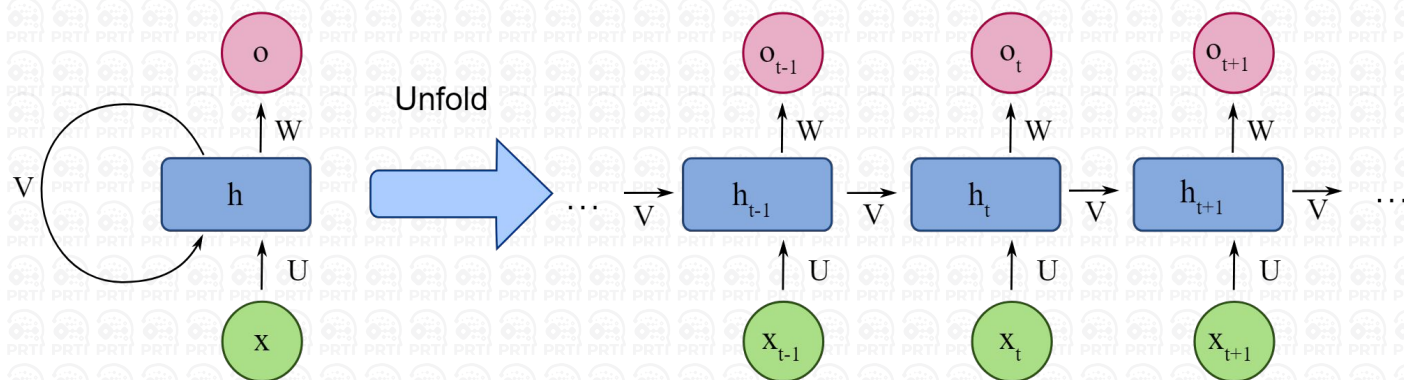
Feature map



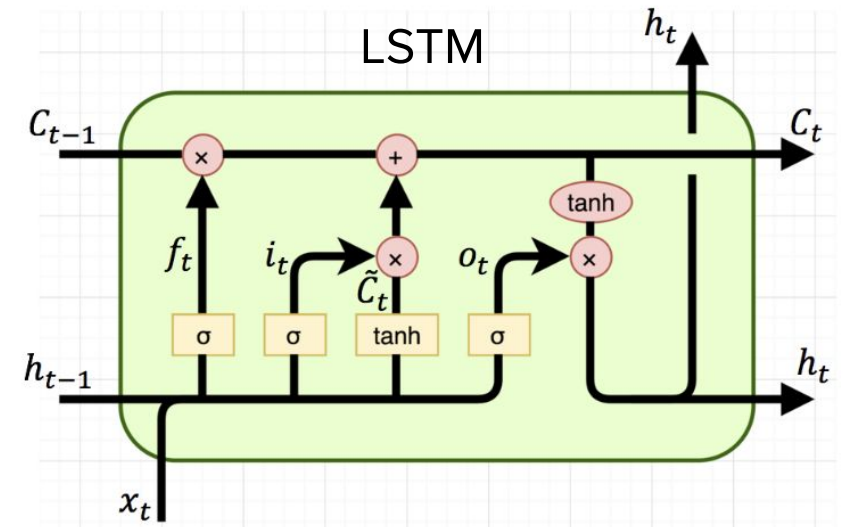
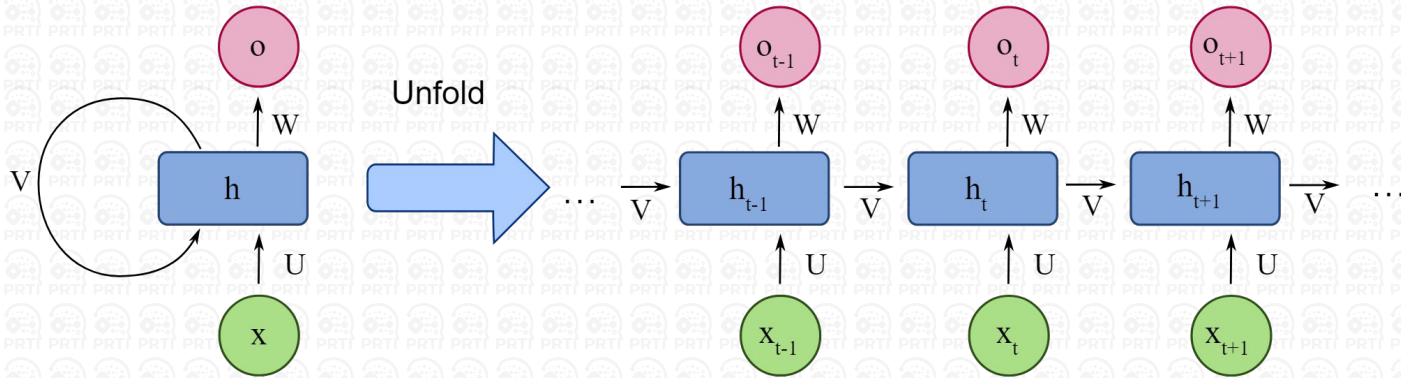
Redes Neurais Convolucionais



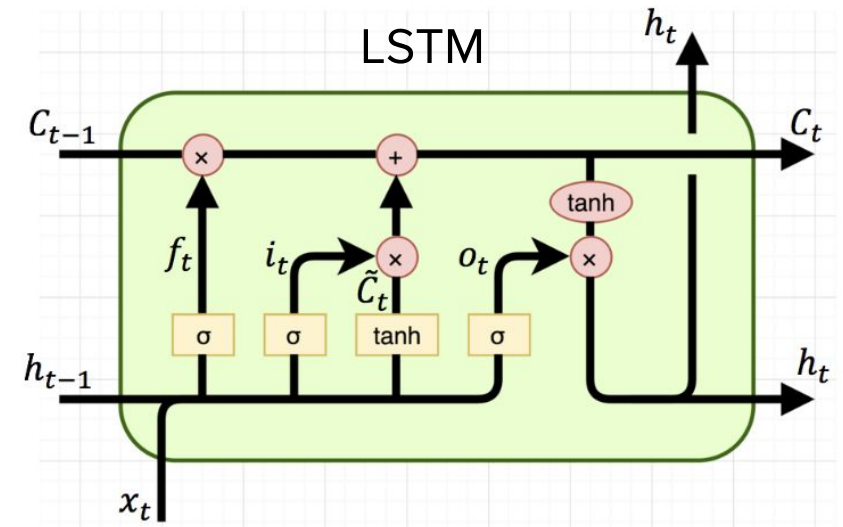
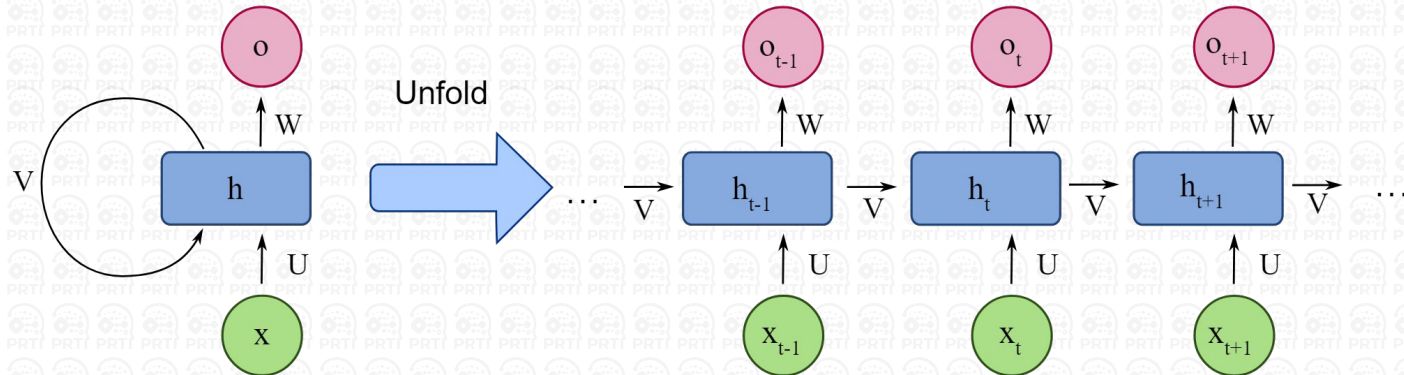
Redes Neurais Recorrentes



Redes Neurais Recorrentes



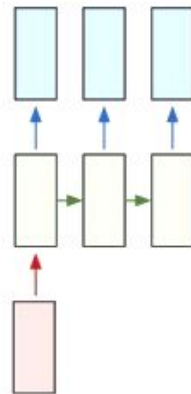
Redes Neurais Recorrentes



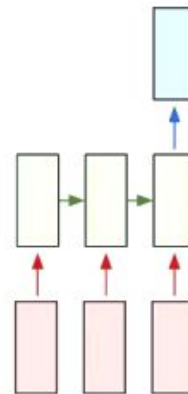
one to one



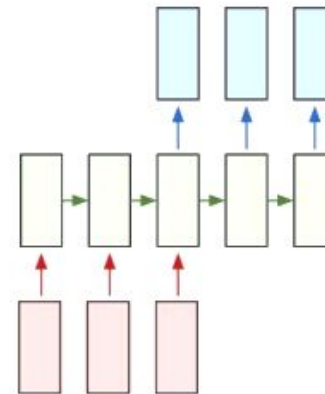
one to many



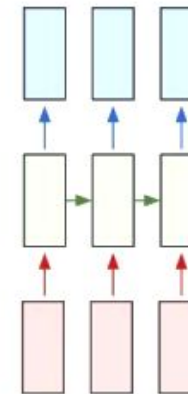
many to one



many to many



many to many





Limitações das Recorrentes e Mecanismo de Atenção



- **Problema das RNNs/LSTMs:** Dificuldade em processar sequências muito longas e impossibilidade de paralelização (o processamento é sequencial, palavra por palavra).
- **A Solução (*Attention*):** Permite que a rede "olhe" para toda a frase de uma vez e aprenda quais palavras são mais relevantes umas para as outras, independentemente da distância.
- **Transformers:** Arquitetura baseada puramente em atenção, dispensando a recorrência e permitindo o treinamento paralelo massivo (base das IAs Generativas atuais).